10^{50}

Number of unique positions in Chess

$3x10^{35}$

Number of years it would take to evaluate Chess

Chess Complexity

Deep Learning for Computer Chess (Part 1)

SCSE21-0010
Manav Arora
U1822077D

Background

Brute Force Engines (BFE)

Algorithm: Fixed Depth Minimax (1928)

Adopted for Chess in 1950

Disadvantages of BFE

Inability to learn

Low computational efficiency and generalisability

Deep Blue vs Kasparov

Engines have been playing at human level for over 20 years.

Humans can't beat modern engines (years of manual feature selection, hardcoded rules and tuning)

Advent of Deep Learning

- Increased Computational Ability
- Shift towards Machine Learning
- Deep Neural Networks -> Human Intuition
- DL Models for Chess: Giraffe, DeepChess, etc.

Deep Learning for Chess

- Giraffe: Deep Learning + Manual feature Selection

  (self play and derives its own rules)
- DeepChess: Deep Learning no a priori knowledge

  (extract features, compare and evaluate positions)

Research Scope

I

Implementation

Train both networks using a plethora of high level chess games

Experiments

Notice the differences between different configurations of both pipelines

II

III

Conclusion

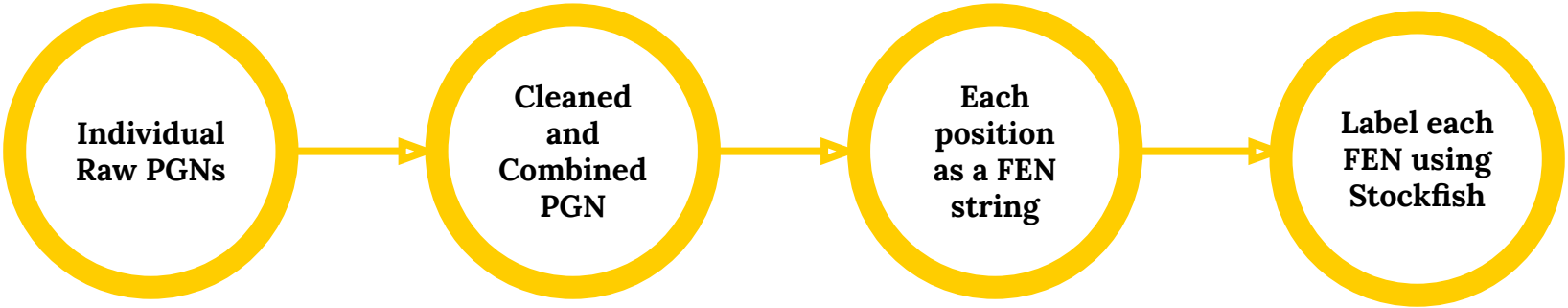
Compare model performance, efficiency and ability to learn human like strategic ideas

Giraffe: Implementation Overview

- Dataset Creation
- Feature Representation
- Data Preparation
- Model Architecture

Giraffe: Dataset Creation

- A dataset of over 10,000 games played at the highest level by five grandmasters (Alexander Alekhine, Garry Kasparov, Anatoly Karpov, Paul Keres, Viktor Korchnoi) was constructed.



* PGN (portable game notation) - stores the moves of the game along with additional information about the game.

* FEN (Forsyth - Edwards Notation) - provides a detailed description of one specific position.

Giraffe: Dataset Creation

Label	Condition on Evaluation
White Winning	Evaluation > 5
White Decisive Advantage	3 < Evaluation <= 5
White Better	1 < Evaluation <= 3
Equal	-1 <= Evaluation <= 1
Black Better	-3 < Evaluation <= -1
Black Decisive Advantage	-5 < Evaluation <= -3
Black Wining	Evaluation < -5

Table 1. Labels to evaluation condition mapping

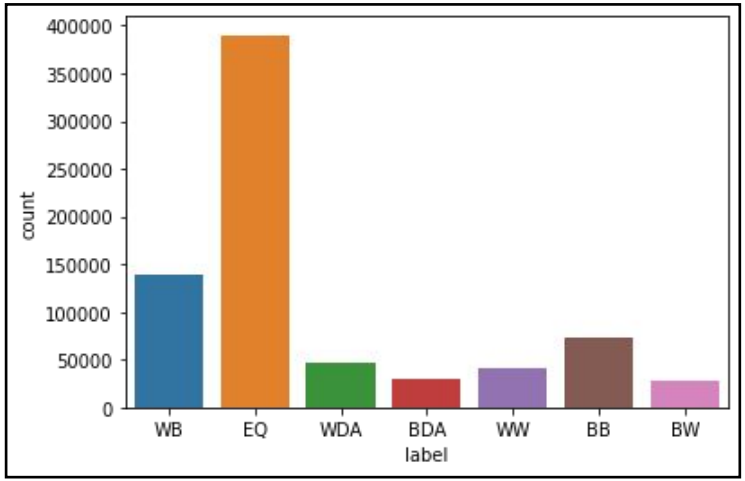


Figure 1. Data distribution of data with respect to labels

Giraffe: Feature Representation

- Allow the network to derive its own rules by self-play.
- Make certain useful features easier to learn.
- This representation amounts to a total of 357 features.

Feature	Description
Side to Move	Whose move it is, i.e either white or black
Castling Rights	Availability of long and short castling rights for either side
Material Configuration	The number of all pieces of different types
Piece Record	Position and presence of each piece on the board
Sliding Motion	The extent to which each sliding piece can move
Game Maps	The lowest valued attacker and defender corresponding to every square

Table 2. Description of features used in Giraffe feature representation

Giraffe: Feature Representation

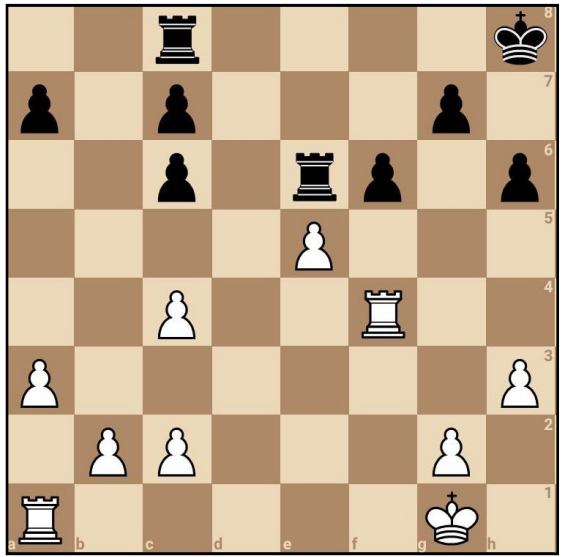


Figure 2. Sample board for feature representation demonstration

Feature	Value
Side to Move	Black
White Long Castle	No
White Short Castle	No
Black Long Castle	No
Black Short Castle	No
White Queens Count	0
White Rooks Count	2
White Bishops Count	0
White Knights Count	0
White Pawns Count	6
…..	……

Table 3. Sample Giraffe feature representation

Giraffe: Data Preparation

- 748617 rows each with 357 features (columns).
- Encode the labels
- Normalize the numerical data
- Split into training and testing sets (90:10)

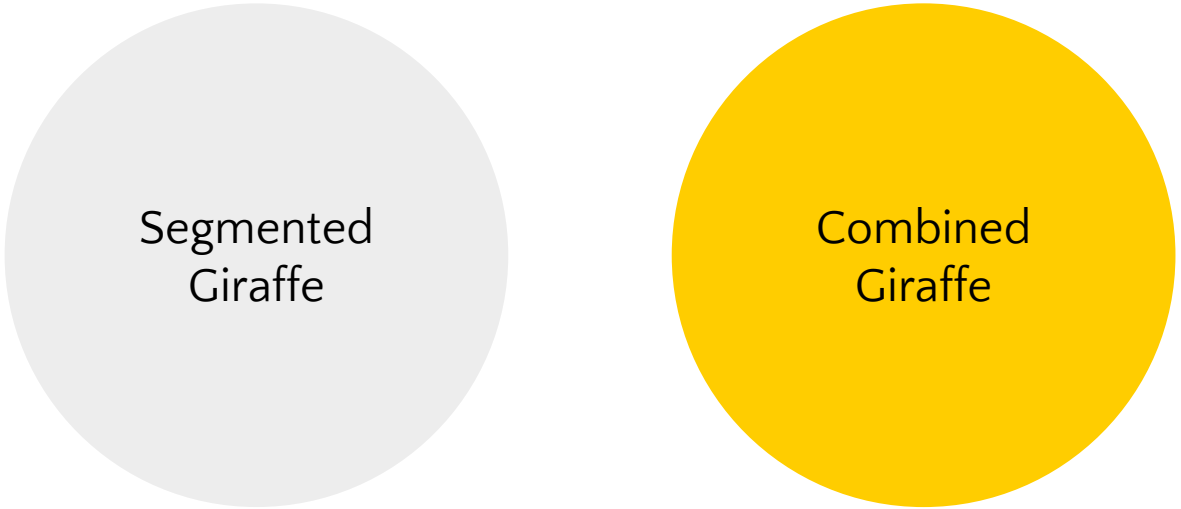
Label	Mapping
Black Better	0
Black Decisive Advantage	1
Black Wining	2
Equal	3
White Better	4
White Decisive Advantage	5
White Better	6

Table 4. Class label to numerical value mapping

Dataset	Shape
X_train	(673755, 356)
X_test	(74862, 356)
Y_train	(673755,)
Y_test	(74862,)

Table 5. Train and test data shapes for Giraffe

Giraffe: Model Architecture



Giraffe: Model Architecture

- Input data is split into three groups that are fed into the network independently.
- Avoid unnecessary connections that bear little to no value and to help prevent overfitting.
- These components are later concatenated to capture interactions between concepts from each individual group.

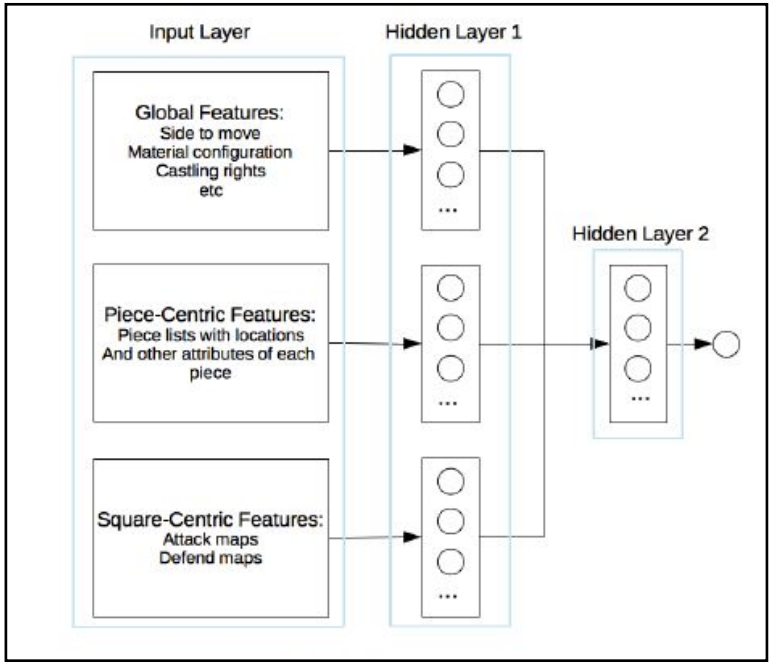


Figure 3. Segmented Giraffe abstract architecture, image courtesy of M. Lai et al.

Giraffe: Model Architecture

- Combined Giraffe omits the input segregation mentioned in the previous approach.
- Establish a point of comparison and to gauge the effects of splitting the input layer.

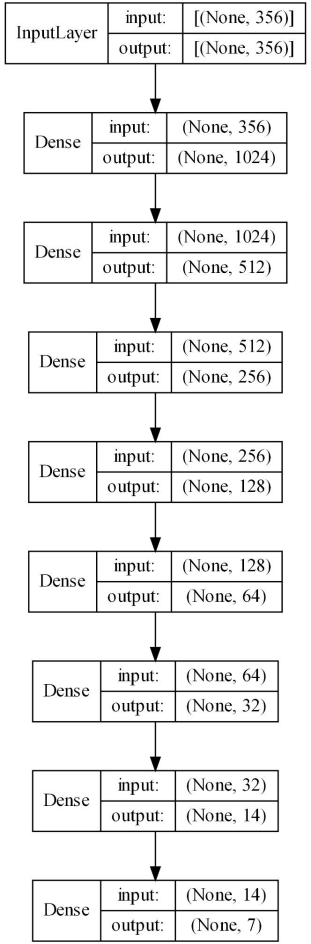


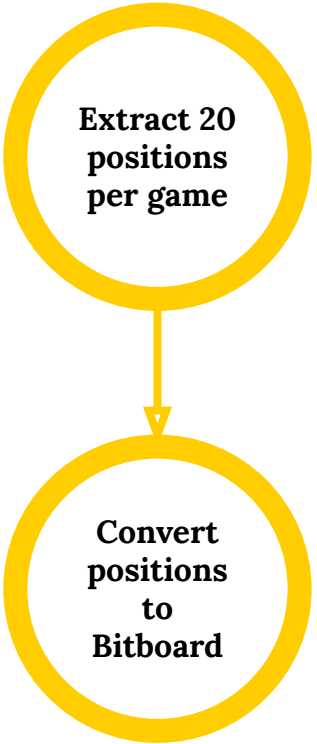
Figure 4. Combined Giraffe complete model architecture

DeepChess: Implementation Overview

- Dataset Creation
- Deep Belief Networks
- Model Architecture

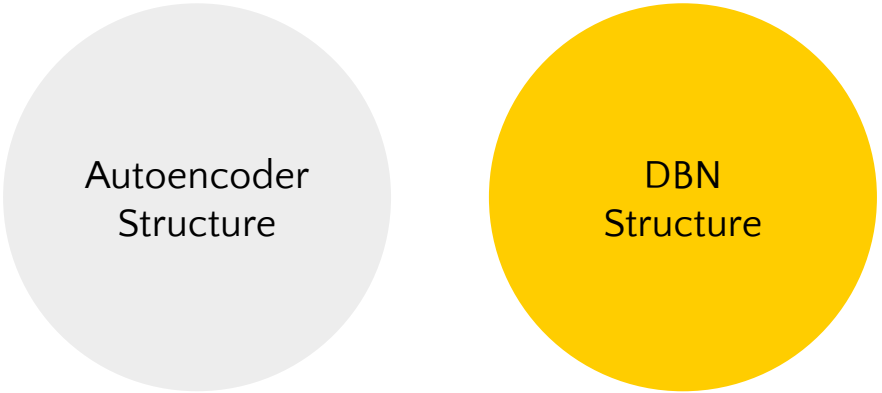
DeepChess: Dataset Creation

- Same games were used for both Giraffe and DeepChess.
- Capture moves, initial moves, drawn games were omitted.
- Each bitboard is a 773-bit vector that encodes information like side to play, piece types, complete 8x8 board representation and castling rights.



DeepChess: Deep Belief Networks

- A Deep Belief Network is a stacked, multilayered network wherein the layers are interconnected. A deep belief network of stacked autoencoders is the core concept behind the DeepChess architecture.



DeepChess: Autoencoder Structure

- An autoencoder follows a bottleneck architecture wherein the first step is to encode the input into a latent low-dimensional code followed by a reconstruction of the input by making use of this latent code. The DeepChess DBN makes use of four autoencoders.

	Encoder	Latent Code	Decoder
Autoencoder I	773	600	773
Autoencoder II	600	400	600
Autoencoder III	400	200	400
Autoencoder IV	200	100	200

Table 6. Configuration of the autoencoders used in the DeepChess DBN

DeepChess: DBN Structure

- The DeepChess autoencoders are stacked such that the next autoencoder is inserted between the latent code and decoder of the previous one. There are two ways such a stacked network can be trained, either by freezing the weights of the previous layer so as to preserve the information they contain for future training rounds or by not freezing the layers and letting them train together.

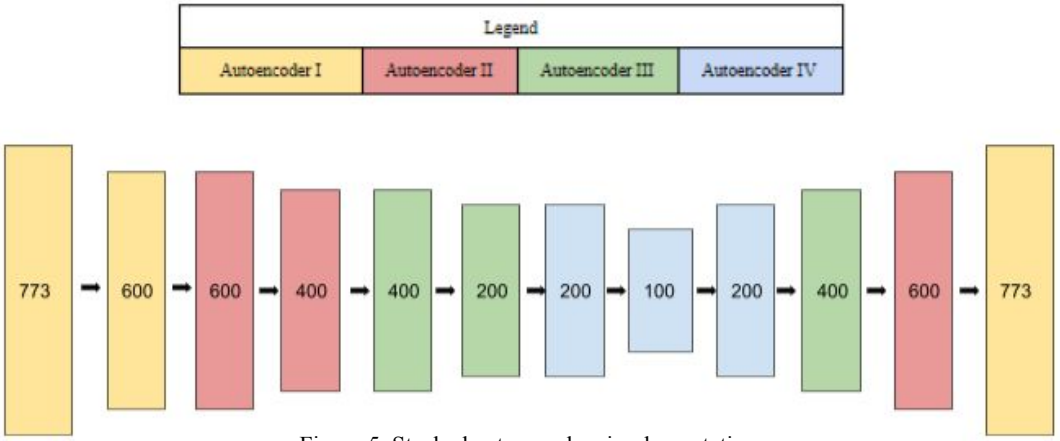


Figure 5. Stacked autoencoders implementation

DeepChess: DBN Structure

- Once the stacked autoencoders were trained, the code layer from each was extracted and stacked to constitute the DeepChess DBN.

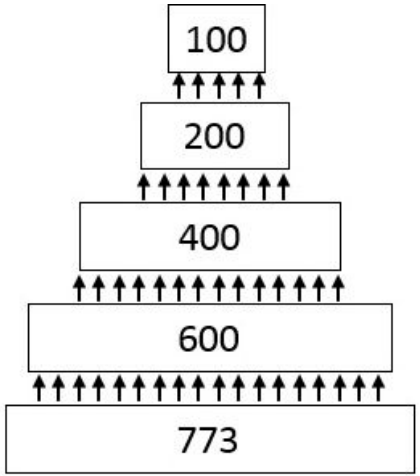


Figure 6. DeepChess DBN architecture, image courtesy of O.E. David et al.

Deepchess: Model Architecture

- Siamese network consisting of two DBNs. Both of which are concatenated and passed through four fully connected layers.
- DBNs -> higher level feature extraction
- Fully connected layers -> evaluate the better position
- Training - 100,000 positions with 50,000 wins for each colour.
- Testing - 10,000 positions with 5,000 wins for each colour.

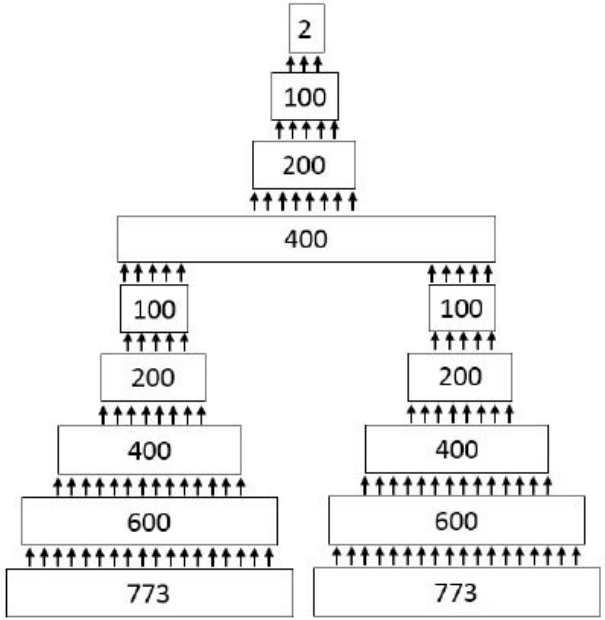


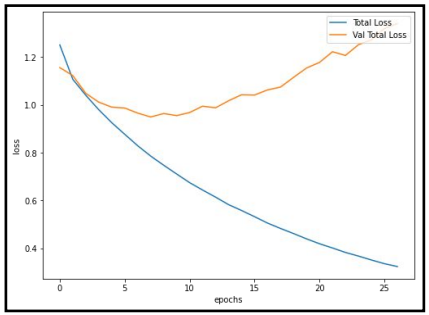
Figure 7. DeepChess architecture, image courtesy of O.E. David et al.

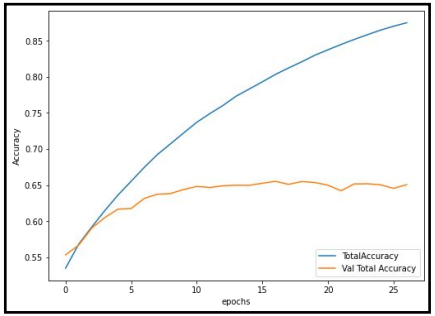
Giraffe: Experiments

- Giraffe was implemented as a 7 class classification problem.
- Parameters:
  - Sparse categorical cross-entropy loss function
  - Adam optimizer
  - Model checkpoint and early stopping callbacks

Giraffe: Experiment 1 - Segmented vs Combined

Segmented Giraffe

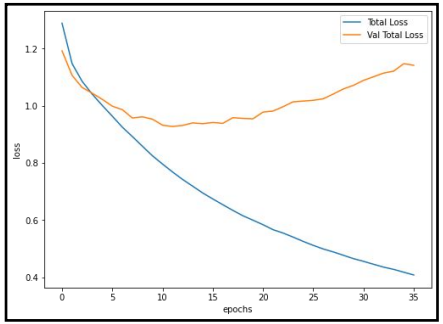


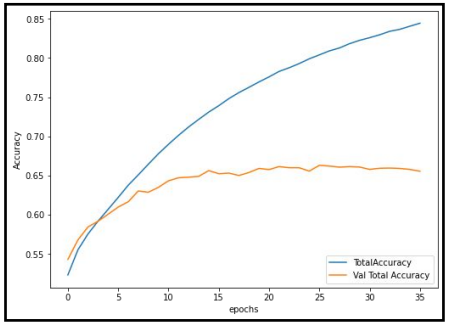


1. Clear signs of overfitting can be observed.
2. Divergent trend exists between the training and test loss and accuracies.
3. The network obtains a maximum test accuracy of 65.5%.

Combined Giraffe

1. Network is plagued by overfitting
2. Marginally outperforms the previous network
3. Accuracy improves from 65.5 % to 66.2 % and the loss decreases from 0.949 to 0.927.





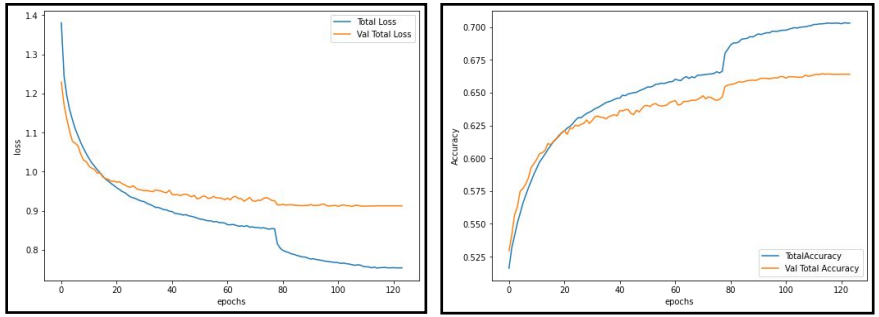
Giraffe: Experiment 1 - Segmented vs Combined

Model	Metric	Value
Segmented Giraffe	Training Accuracy	0.874
	Testing Accuracy	**0.655**
	Training Loss	0.323
	Testing Loss	**0.949**
Combined Giraffe	Training Accuracy	0.844
	Testing Accuracy	**0.662**
	Training Loss	0.407
	Testing Loss	**0.927**

Table 7. Segmented and Combined Giraffe performance summary

Giraffe: Experiment 2 - Regularisation (Dropout)

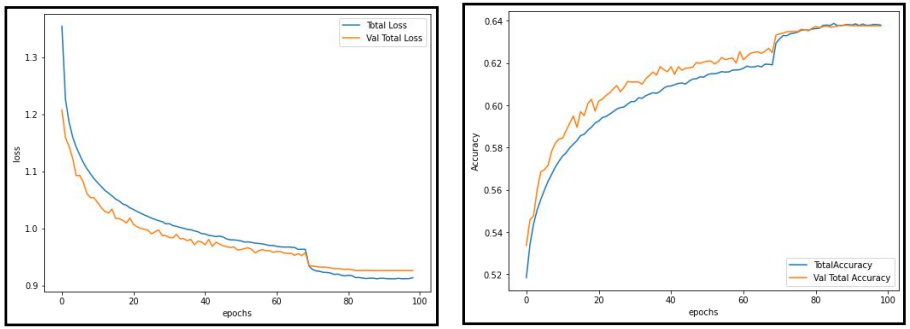
Reg. Segmented Giraffe



1. The network trains well without overfitting
2. Outperforms its non-regularized implementation by obtaining a testing accuracy of 66.4%
3. Show that regularization has a positive effect

Reg. Combined Giraffe

1. Network performance drops from 66.2% to 63.7%.
2. Dropout neurons had a significant contribution to performance
3. Either a small dropout ratio or other forms of regularization could be helpful.



Giraffe: Experiment 2 - Regularisation (Dropout)

Model	Metric	Value
Segmented Regularized Giraffe	Training Accuracy	0.703
	Testing Accuracy	**0.664**
	Training Loss	0.753
	Testing Loss	**0.910**
Combined Regularized Giraffe	Training Accuracy	0.638
	Testing Accuracy	**0.637**
	Training Loss	0.911
	Testing Loss	**0.925**

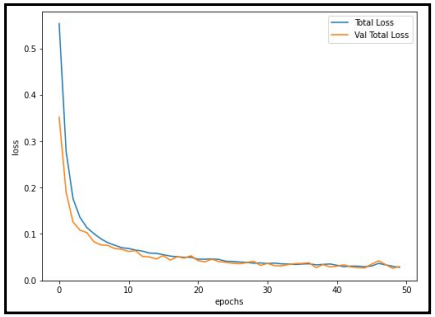
Table 8. Segmented Regularized and Combined Regularized Giraffe performance summary

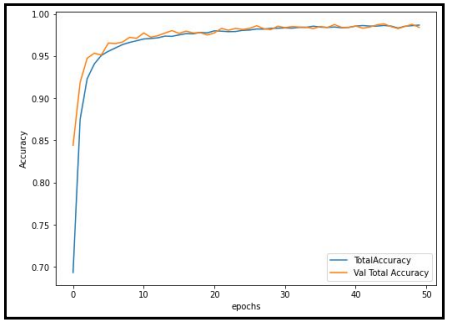
DeepChess: Experiments

- DeepChess was implemented as a binary classification problem.
- Parameters:
  - The input comprised 100,000 randomly sampled positions with an equal win split between the two colours.
  - Rectified linear units activation
  - Base learning rate of 0.005 and decreased by 1% every epoch.

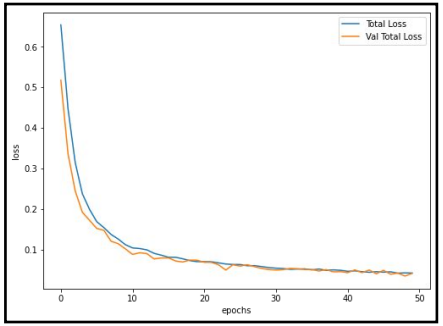
DeepChess: Experiment 1 - Effect of Optimizer and Loss Function

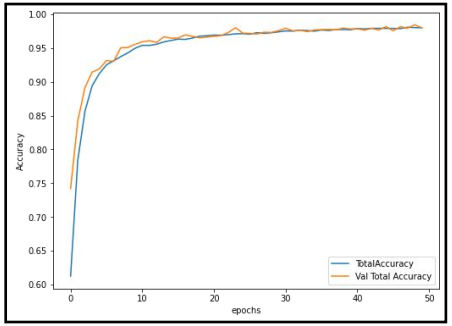
Adam + BCE DeepChess





SGD + MSE DeepChess





Having observed the results of both these networks it is quite discernable that the DeepChess model trained with adam and binary cross-entropy outperforms the other network across all metrics. Thus moving forward, all models will make use of these parameters.

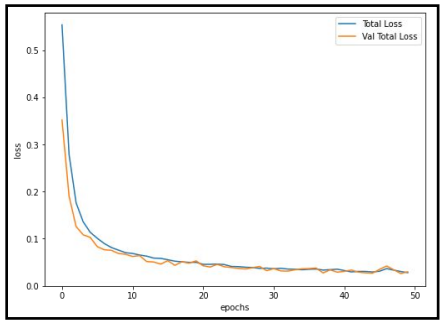
DeepChess: Experiment 1 - Effect of Optimizer and Loss Function

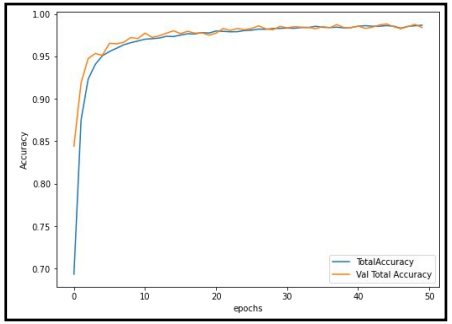
Model	Metric	Value
DeepChess with DBN trained using Adam and Binary Cross Entropy	Training Accuracy	0.986
	Testing Accuracy	**0.988**
	Training Loss	0.027
	Testing Loss	**0.026**
DeepChess with DBN trained using SGD and MSE loss	Training Accuracy	0.980
	Testing Accuracy	**0.984**
	Training Loss	0.042
	Testing Loss	**0.034**

Table 9. DeepChess with DBN trained using different sets of parameters performance summary

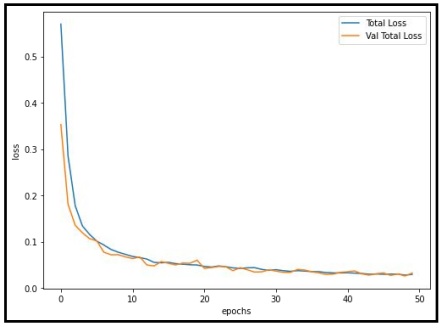
DeepChess: Experiment 2 - Effect of Freezing Decoder Layers

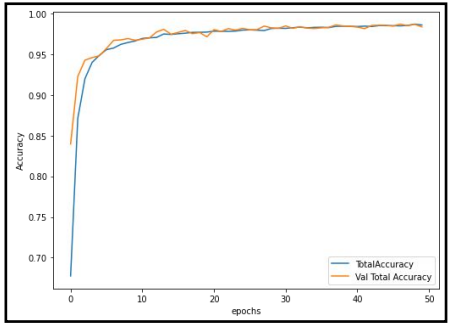
No Freeze DBN DeepChess





Frozen DBN DeepChess





Having observed the results of both these networks, it is fair to say that the results are extremely similar to each other. This implies that even after freezing certain layers in the DBN training process the overall DeepChess model was able to maintain its performance. Therefore, in such a situation, the network with the frozen layers is preferable as it takes relatively less time to train and is able to optimize the performance while only employing a subset of the feature space.

DeepChess: Experiment 2 - Effect of Freezing Decoder Layers

Model	Metric	Value
DeepChess with DBN trained without freezing decoder layers	Training Accuracy	0.986
	Testing Accuracy	**0.988**
	Training Loss	0.027
	Testing Loss	**0.026**
DeepChess with DBN trained with freezing decoder layers	Training Accuracy	0.987
	Testing Accuracy	**0.987**
	Training Loss	0.027
	Testing Loss	**0.026**

Table 10. DeepChess with DBN trained with and without freezing decoder layers performance summary

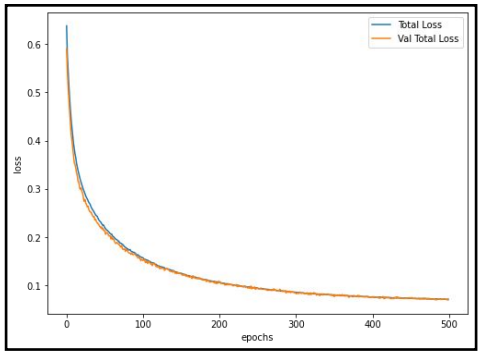
DeepChess: Experiment 3 - Bigger Dataset (LiChess)

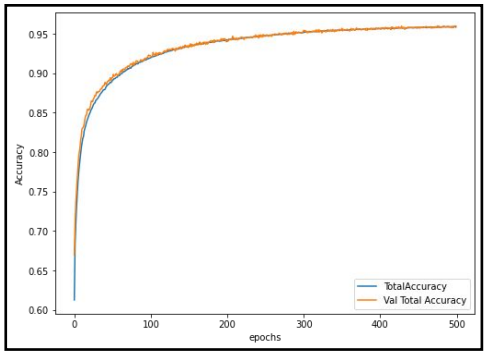
- Thousands if not millions of samples -> a dependable and generalized model.
- Even though we are able to obtain great performance, these networks' ability to generalize may be compromised due to a smaller test set. (Draws)
- For a more dependable and generalizable implementation of DeepChess, a data set of over 100,000 games was created from games played on Lichess (2200+).
- Training - 1,000,000 positions with 500,000 wins for each colour.
- Testing - 100,000 positions with 50,000 wins for each colour.

DeepChess: Experiment 3 - Bigger Dataset (LiChess)

This network achieves a testing accuracy of 96% and a testing loss of 0.69.

This is a much more realistic estimation of DeepChess architecture's true performance as this network has been tested over a much larger and diverse test set as compared to the previous implementations of DeepChess.





Model	Metric	Value
LiChess - DeepChess with DBN trained with freezing decoder layers	Training Accuracy	0.959
	Testing Accuracy	**0.960**
	Training Loss	0.070
	Testing Loss	**0.069**

Conclusion

- Giraffe
  - Accuracy of 66.4% with the segmented regularized architecture.
  - Makes use of deep learning while still making use of a custom feature representation.
- DeepChess
  - Accuracy of 98.8% on the smaller dataset and an accuracy of 96% on the larger LiChess dataset.
  - Evaluates positions using a deep neural network without any a priori knowledge of the rules of chess.

Two deep learning pipelines were implemented to create evaluation functions that can learn the rules and intricacies of the chess by themselves from the database and can evaluate, and analyze chess more efficiently by comprehending complex strategic ideas that are intuitive to humans but have been difficult for computers to grasp for a long time.

Conclusion: Future Research

- Combine the feature representation from Giraffe into the network architecture from DeepChess.
- Extensive hyperparameter tuning.
- The training was restricted for DeepChess (LiChess) due to computational limitations.

Thanks!

Any questions ?

You can find me at

- manav004@e.ntu.edu.sg
- +65 9133 2470